

Ionic Conductivity and Nernst-Einstein Specification

C2 Green-Kubo Integration, Explicit Plateau Estimation, and Compatible Self-Diffusion Comparison

mdstats

2026-07-22

Contents

Purpose and status	1
Physical definitions	1
Preconditions	2
Public APIs	2
Running Green-Kubo integration	2
Explicit conductivity plateau	2
Nernst-Einstein comparison	3
Result schemas	3
IonicConductivityResult	3
IonicConductivityEstimate	4
NernstEinsteinComparisonResult	4
Failure policy	4
Required tests	4
References	5

Purpose and status

This document specifies roadmap stage **C2** for `mdstats 0.19.86a0`. C2 consumes the neutral collective-current correlations implemented by C0-C1 and adds:

1. three-dimensional isotropic Green-Kubo conductivity integration;
2. explicit interval-based conductivity plateau estimation; and
3. a fail-closed Nernst-Einstein comparison against compatible species self-diffusion estimates.

The implementation resides in `mdstats.analysis.ionic_conductivity`. The Green-Kubo relation is established linear-response theory [1, 2]. The Nernst-Einstein relation is the independent-particle combination of Einstein self diffusion with charge transport [3]. Composite trapezoidal quadrature is provided by the package’s validated SciPy-backed quadrature primitive [4]. Plateau selection, result schemas, compatibility checks, ratio policy, and immutability are `mdstats` design decisions.

Physical definitions

For a neutral system with microscopic charge current

$$\mathbf{J}_q(t) = \sum_i q_i \mathbf{v}_i(t),$$

the three-dimensional isotropic Green-Kubo conductivity is

$$\sigma(t) = \frac{1}{3Vk_{\text{B}}T} \int_0^t \langle \mathbf{J}_q(0) \cdot \mathbf{J}_q(\tau) \rangle d\tau.$$

C1 stores current correlations in $e^2 \text{ Angstrom}^2/\text{ps}^2$. After integration over picoseconds, the canonical integral has units $e^2 \text{ Angstrom}^2/\text{ps}$. The exact numerical conversion used by C2 is

$$\sigma [\text{S/m}] = I [e^2 \text{Angstrom}^2/\text{ps}] \frac{e^2 10^{22}}{3V k_B T},$$

where e is the elementary charge in coulombs, V is supplied in Angstrom³, and T is in kelvin. The factor 10^{22} follows from Angstrom²/ps = $10^{-8} \text{ m}^2/\text{s}$ and Angstrom⁻³ = 10^{30} m^{-3} .

For exact current groups a, b , C2 integrates every ordered contribution

$$\sigma_{ab}(t) = \frac{1}{3V k_B T} \int_0^t C_{ab}(\tau) d\tau,$$

without symmetrizing a, b . The invariant is

$$\sigma(t) = \sum_{a,b} \sigma_{ab}(t).$$

For group populations N_a , uniform group charges $z_a e$, and compatible three-dimensional self-diffusion estimates D_a , the Nernst-Einstein estimate is

$$\sigma_{\text{NE}} = \frac{e^2}{V k_B T} \sum_a N_a z_a^2 D_a.$$

Preconditions

C2 conductivity integration is deliberately restricted to the contract that can be interpreted without additional cell-deformation theory:

- the input is a `CurrentCorrelationResult`;
- all three periodic-axis flags are true;
- the full cell matrix is fixed across the analyzed frames;
- the result signature uses the full three-dimensional physical subspace;
- temperature and volume are finite and strictly positive; and
- an explicit volume assertion, when supplied, agrees with stored fixed-volume provenance within the C0 cell-equivalence tolerance.

A variable cell is rejected before any explicit volume argument is considered. A scalar override is an assertion, not a replacement for incompatible source provenance.

Public APIs

Running Green-Kubo integration

```
def integrate_ionic_conductivity(
    correlation: CurrentCorrelationResult,
    *,
    temperature_k: float,
    volume_a3: float | None = None,
    maximum_time_ps: float | None = None,
) -> IonicConductivityResult:
    ...
```

The function retains a prefix of the stored lag grid, performs cumulative composite trapezoidal integration with an exact leading zero, and converts the total and every ordered group-pair contribution to SI conductivity. It does not smooth, extrapolate, fit a long-time tail, or choose a plateau.

Explicit conductivity plateau

```
def estimate_ionic_conductivity_plateau(
    running: IonicConductivityResult,
    *,
    time_range_ps: tuple[float, float],
```

```

    minimum_points: int = 8,
    slope_tolerance_s_per_m_ps: float | None = None,
) -> IonicConductivityEstimate:
    ...

```

The selected stored samples must be uniformly spaced. The estimate is the arithmetic mean of the running conductivity over the explicit interval. The function records centered linear-fit slope, intercept, residual, span, endpoint drift, and optional slope-stability status. It does not claim an independent sample standard error from one serially correlated running curve.

The same interval is applied independently to every ordered group-pair running contribution. Their interval means must sum to the total estimate.

Nernst-Einstein comparison

```

def compute_nernst_einstein_comparison(
    conductivity: IonicConductivityEstimate,
    species_diffusion: Mapping[str, DiffusionEstimate],
    *,
    temperature_k: float | None = None,
    volume_a3: float | None = None,
) -> NernstEinsteinComparisonResult:
    ...

```

The conductivity estimate must contain a nonempty exact group partition. The mapping keys must match group_names exactly and in the same order. Each group must have one uniform nonzero charge. The matching diffusion estimate must:

- contain a complete DynamicsInputSignature;
- select exactly that group's canonical atom indices;
- use the same trajectory fingerprint, frames, times, sample spacing, source files, and drift-reference population;
- use the full three-dimensional subspace; and
- have a finite nonnegative diffusion coefficient.

Coordinate construction and velocity-source labels are allowed to differ, because one side is displacement based and the other velocity based. Variable cell comparisons are impossible because the conductivity estimate cannot be constructed from variable-cell provenance.

Optional temperature and volume arguments are consistency assertions. Counts and charges are derived from the conductivity provenance and are never supplied a second time.

Result schemas

IonicConductivityResult

The immutable running result stores:

lag_steps	(L,)
lag_times	(L,)
scalar_correlation_e2_a2_per_ps2	(L,)
integrated_correlation_e2_a2_per_ps	(L,)
running_conductivity_s_per_m	(L,)
group_names	tuple[str, ...]
group_scalar_correlation_e2_a2_per_ps2	(L, G, G) or None
group_integrated_correlation_e2_a2_per_ps	(L, G, G) or None
group_running_conductivity_s_per_m	(L, G, G) or None
temperature_k	scalar
volume_a3	scalar
conductivity_prefactor	scalar
pbcs	(3,), all true
cell_mode	"fixed"
fixed_volume_a3	scalar
total_charge_e	scalar
neutrality_tolerance_e	scalar
charges_e	(N,)

current_atom_indices	(M,)
group_atom_indices	exact immutable mapping
signature	DynamicsInputSignature
metadata	recursively immutable mapping

The constructor rechecks quadrature, SI conversion, ordered group sums, charge identity, fixed-volume provenance, and the full three-dimensional subspace.

IonicConductivityEstimate

The explicit estimate stores the scalar interval mean, selected interval, number of points, optional slope-stability decision, complete diagnostics, ordered group-pair interval means, temperature, volume, full periodic-axis and fixed-cell provenance, neutrality state, charge/group identity, and the original dynamics signature.

NernstEinsteinComparisonResult

The comparison stores:

- collective and Nernst-Einstein conductivities;
- signed difference collective - Nernst-Einstein;
- absolute difference;
- both directional ratios;
- explicit ratio-defined flags;
- counts, uniform group charges, diffusion values, and per-group NE contributions; and
- the summed off-diagonal ordered group-pair conductivity contribution.

A ratio whose denominator is zero is stored as NaN, with its corresponding boolean flag false. No universal Haven-ratio label is assigned because the two reciprocal conventions are both used in the literature.

Failure policy

C2 fails closed for:

- non-C1 inputs;
- nonpositive or non-finite temperature or volume;
- partial periodicity;
- variable full-cell-matrix provenance;
- an inconsistent volume assertion;
- malformed or nonmonotone lag grids;
- an out-of-range truncation time;
- a plateau interval outside the retained range;
- too few or nonuniformly spaced plateau samples;
- missing current groups for Nernst-Einstein comparison;
- group/diffusion key, atom-selection, trajectory, frame, timing, drift, or projection mismatch;
- a mixed-charge current group;
- a missing or negative species diffusion estimate;
- malformed result construction; and
- any total/group, quadrature, SI-conversion, or ratio identity failure.

Required tests

C2 acceptance requires tests for:

1. exact SI conversion against fundamental constants;
2. cumulative trapezoidal integration of an analytic sampled correlation;
3. inverse temperature and volume scaling;
4. truncation to a requested maximum time;
5. ordered group-pair integration and exact total sum;
6. partial-periodic, variable-cell, and inconsistent-volume rejection;
7. signature and charge/group provenance preservation;

8. explicit plateau mean and diagnostics;
9. plateau group-pair sum and nonuniform-grid rejection;
10. independent-particle synthetic Nernst-Einstein agreement;
11. controlled collective enhancement and suppression;
12. every group, atom-selection, trajectory, frame, drift, rank, charge, and thermodynamic-state mismatch;
13. zero-denominator ratio policy;
14. deep array, mapping, and nested-metadata immutability;
15. public exports; and
16. regression compatibility with the complete existing dynamics branch.

References

- [1] M. S. Green, “Markoff Random Processes and the Statistical Mechanics of Time-Dependent Phenomena. II. Irreversible Processes in Fluids,” *Journal of Chemical Physics* **22**, 398-413 (1954). DOI: 10.1063/1.1740082.
- [2] R. Kubo, “Statistical-Mechanical Theory of Irreversible Processes. I. General Theory and Simple Applications to Magnetic and Conduction Problems,” *Journal of the Physical Society of Japan* **12**, 570-586 (1957). DOI: 10.1143/JPSJ.12.570.
- [3] A. Einstein, “Über die von der molekularkinetischen Theorie der Wärme geforderte Bewegung von in ruhenden Flüssigkeiten suspendierten Teilchen,” *Annalen der Physik* **322**, 549-560 (1905). DOI: 10.1002/andp.19053220806.
- [4] P. Virtanen, R. Gommers, T. E. Oliphant, et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods* **17**, 261-272 (2020). DOI: 10.1038/s41592-019-0686-2.